

CS7CS4/CSU44061 Machine Learning

Student Name: Stephen Komolafe

Student Number: 21336975

Week 9 Assignment – Generative Pre-trained Transformer Configuration & Evaluation

Part I - Configuration:

(a) Dataset - input childSpeech trainingSet.txt

The training dataset contains short, simple sentences representative of early child speech. Many phrases resemble how toddlers and young children in general talk, with examples such as, “*I dance Look airplane*”, “*Daddy read me dis I wraw*”, “*Mama I wan*”. The generated text consists mostly of short English words, childlike word forms (“*Mama*”, “*Daddy*”), spaces, and line breaks. There is no punctuation or numerical content. The dataset contains 246,982 tokens and a vocabulary size of 40, consisting mostly of alphabetic characters letters (both uppercase and lowercase), spaces, and newline characters.

Dataset - input childSpeech testSet.txt

The test dataset follows the same structure and grammatical style as the training set, containing elementary sentences typical of young children’s speech. Examples phrases include “*Yummy apple Mama I want play with big red ball outside*”, “*I hide I make mess*”, and “*No like it Look moon*”. The language maintains the same childlike language characteristics as the training set, including grammatical errors and a lack of punctuation. The dataset contains 24,113 tokens and shares a vocabulary size of 40, matching the training set. As with training data, it is composed exclusively of alphabetic characters, spaces, and newline symbols.

Dataset - input shakespeare.txt

This dataset contains text from Shakespeare’s plays and is notably larger and more complex than the child-speech train and test datasets. It includes dialogue, stage directions, and archaic English vocabulary commonly found in Elizabethan literature. The language contains a full range of punctuation such as commas, semicolons, colons, exclamation points, and question marks. Example generated text includes “*We the mudder of what the laws ignoranty poor suffeence?*”, “*The other's god, I'll have togue.*”, and “*Not, what though mine is thy grace? I will send thee when thousands,*”. The dataset contains 1,115,394 tokens and a vocabulary size of 65, covering upper- and lowercase letters as well as a variety of punctuation symbols. Compared to the child-speech datasets, this dataset’s generated Shakespearean text is far richer in structure, and more varied structure, with longer sentences, complex syntax, and stylistic diversity.

- (b) To reduce the model to fewer than one million parameters, the hyperparameters `n_embd`, `n_head`, and `n_layer` were modified, as they have the largest influence on total parameter count. These three values determine dimensionality of the embedding space, the number of attention heads, and the depth of the network, respectively. Because both the self-attention and feed-forward components scale linearly or quadratically with the embedding dimension, adjusting these parameters yields the most substantial reductions in model size.

The embedding dimension (`n_embd`) was reduced from 384 to 64. this value affects nearly all weight matrices in the transformer’s architecture, including token embeddings, positional embeddings, attention projections, and all feed-forward layers; downsizing this value would have the largest decrease in model size. To ensure compatibility with the smaller embedding dimension across attention heads, `n_head` was

lowered from 6 to 4, preserving an even division of head dimensionality ($64 / 4 = 16$). This maintains stable multi-head attention behaviour while reducing the number of projection matrices and associated parameters.

The number of transformer blocks (**n_layer**) was reduced from 6 to 2. Each additional layer introduces another full attention module and feed-forward network, so **n_layer** scales the parameter count almost linearly. Reducing model depth therefore provides another significant decrease in model size without altering its architecture.

In contrast, hyperparameters such as **batch_size**, **learning_rate**, **dropout**, and **block_size** were left unchanged because they do not contribute to the number of trainable parameters. While they influence aspects such as optimisation behaviour, training speed, and memory usage, they do not influence the size of the model's weight matrices.

By selectively adjusting only the hyperparameters that directly shape the model architecture, the total parameter count was reduced from 10.77 million to 0.121256 million. This downsized configuration trains significantly faster and remains expressive enough to learn meaningful patterns from the child-speech dataset.

- (c) Two additional downsized configurations were created to examine how model size influences learning quality. The first configuration (see i(b)) used a significantly reduced embedding dimension and minimal depth, resulting in a total of 0.121256 million parameters. The second configuration kept the same embedding dimension, **n_embd** = 64, but increased the number of layers (**n_layer** = 4, **n_head** = 4), giving the model more depth while remaining under the one million parameter limit. The third configuration preserved depth while further reducing width (**n_embd** = 32, **n_layer** = 2, **n_head** = 4), forming a very narrow model with greatly reduced representational capacity. All three model configurations were trained for 2000 iterations on the `input_childSpeech_trainingSet.txt` dataset. The losses and generated output for each configuration were recorded as shown below.

Configuration 1:

Step	Train Loss	Validation Loss	Generated Output (first 6 lines out of 22)
0	3.7307	3.7311	Go park No mine
500	0.4842	0.4892	Dadddy read book moore bed I jump
1000	0.3616	0.3633	I sing Shoes on
1500	0.3471	0.3490	Bath time Go park
1999	0.3415	0.3445	Whe ere kitd ball Go park Noo stop Shufokogy I p danck

Table 1. Train & Validation Loss at 500 iterations intervals + Generated Output Excerpt (Configuration 1)

Despite its small size, the first configuration, which contained 0.121256 million parameters, performed adequately. The validation loss remained nearly identical to the training loss throughout all iterations, indicating little to no overfitting. This suggests that the model generalises well to unseen data within the child-speech domain. In terms of quality, the generated output is mostly coherent and structurally similar to phrases found in the dataset. Many of the lines resemble typical child language, such as “Go park No mine”, “I sing Shoes on”, and “Bath time Go park”. Although there are occasional misspellings and pseudo-words (“kitd”, “Shufokogy”, “I p danck”), these errors are consistent with the noisy, irregular nature of the dataset and reflect the model's ability to mimic childlike phonetic spelling.

Configuration 2:

Step	Train Loss	Validation Loss	Generated Output (first 6 lines out of 22)
0	3.6824	3.6835	I want more juiceable Al gone Shoes on Uh oh spill More more Mama I waken more juice please More bubbles I want more jump fast Look moon Look moon Daddy play I want more juice please
500	0.4965	0.4991	
1000	0.3533	0.3547	
1500	0.3404	0.3431	
1999	0.3359	0.3378	

Table 2. Train & Validation Loss at 500 iterations intervals + Generated Output Excerpt (Configuration 2)

The second configuration, at 0.22084 million parameters, achieved the lowest validation loss of the three models (0.3378) and maintained a minimal gap between training and validation losses, indicating little to no overfitting. This model's increased capacity appears to improve both consistency and contextual recall in generated sequences. The generated output is noticeably more fluent and semantically stable than in Configuration 1, producing longer, more structured phrases such as *"More bubbles I want more jump fast"*, *"Daddy play I want more juice please"*, and repeated childlike expressions such as *"Look moon Look moon."* The repetition, vocabulary usage, and short multi-phrase expressions are consistent with the style of the training set. Compared to the other configurations, this model produces the cleanest and most interpretable output, suggesting that the increased capacity allowed it to capture more nuanced distributional patterns without overfitting.

Configuration 3:

Step	Train Loss	Validation Loss	Generated Output (first 6 lines out of 22)
0	3.6926	3.6927	Big coone I wash h hands Shoes big fluffffy doggy at park it I want mess No mine Daddy play Read buoook I found it Uh oh spillll b
500	1.5402	1.5417	
1000	0.5401	0.5425	
1500	0.4302	0.4321	
1999	0.3964	0.3991	

Table 3. Train & Validation Loss at 500 iterations intervals + Generated Output Excerpt (Configuration 3)

The third and smallest configuration, with only 0.036 million parameters, had a significantly higher validation loss (0.3991), as expected for a model with limited representational capacity. While it did not overfit, the validation loss was closely similar to the training loss; it learnt a less detailed approximation of the dataset. The generated output includes more fragmentary and unstable constructions, such as *"Big coone"*, *"Shoes big fluffffy doggy at park it"*, and *"Read buoook I found it Uh oh spillll b"*, alongside more reasonable phrases like *"I wash h hands"*, *"I want mess"*, and *"No mine Daddy play"*. The model often drifts into truncated or semi-random letter sequences and shows a limited ability to maintain structure across phrases. This behaviour reflects the model's constrained depth and embedding size: it can capture high-level patterns, for example, common verbs and simple two-word commands, but struggles with coherence, phrase boundaries, and consistent vocabulary usage. Although the model does learn meaningful patterns, its output is noticeably noisier and less fluent than the larger configurations.

- (d) In the transformer's self-attention mechanism, the input embeddings are projected into query (Q), key (K), and value (V) vectors through linear layers. These linear layers can include bias terms, which serve as additive constants applied after matrix multiplication. Although adding biases increases the number of parameters slightly, it gives the model more flexibility in shaping the Q, K, and V representations.

Including bias terms allows the model to shift the projected vectors in ways that are independent of the input token. Without biases, the model can only express relationships as weighted combinations of input embeddings, whereas with biases, the attention mechanism can introduce baseline offsets that influence attention scores even when two tokens have similar embeddings, helping the model to capture

nonlinear, asymmetric, or position-specific patterns. In a dataset like child speech, where tokens such as "I", "No", "More", and "Mama" frequently appear in similar positions, learnt offsets make it easier for the model to represent habitual linguistic tendencies.

Bias terms also contribute to more stable optimisation. Early in training, when weight matrices are still close to their initial values, biases ensure that Q, K, and V projections produce non-zero outputs. In turn, it allows the attention mechanism to break symmetry sooner, preventing the model from falling into uniform or degenerate attention states.

- (e) Skip connections are an essential part of transformer architectures, as they stabilise the training of deep models. By adding each sublayer's input back to its output, the model only needs to learn a small residual transformation rather than reconstructing the entire representation from the beginning. This prevents vanishing gradients and allows information to flow smoothly through multiple layers, leading to faster and more reliable convergence.

Skip connections also help in preserving valuable linguistic patterns instead of overwriting them at every layer. For child-speech modelling, simple recurring structures like "I want..." or "More ..." can pass through unchanged while deeper layers apply only the refinements needed. Even in small models, this results in better generalisation and more coherent text generation. Without skip connections, the model would become more difficult to train, unstable as depth increased, and perform worse overall.

Part II - Evaluation:

To evaluate the trained model on the test sets, an evaluation function, `eval_on_testset`, was added to compute the average loss over a given text file:

```
@torch.no_grad()
def eval_on_testset(path):
    # load file
    with open(path, 'r', encoding='utf-8') as f:
        test_text = f.read()
    # encode
    test_data = torch.tensor(encode(test_text), dtype=torch.long)

    losses = []
    model.eval()
    # break data into blocks of size block_size+store loss of each block
    for i in range(0, len(test_data) - block_size, block_size):
        x = test_data[i:i+block_size].unsqueeze(0).to(device)
        y = test_data[i+1:i+block_size+1].unsqueeze(0).to(device)
        _, loss = model(x, y)
        losses.append(loss.item())
    # retrain+return average loss
    model.train()
    return sum(losses) / len(losses)
```

The function loads the test file, encodes it into integer token IDs, and iterates through the sequence in non-overlapping blocks of size `block_size`. For each block, the model predicts the next-token distribution and computes the loss against the true next characters. The function stores these losses for every block, averages them, and returns the mean test loss. The model is put into evaluation mode (`model.eval()`) to disable dropout and ensure deterministic outputs. After all blocks are processed, it is switched back to training mode (`model.train()`) so that additional training or text generation can proceed normally.

A means to evaluate the child-speech test set on a dummy model was also implemented as a baseline to establish a reference point for comparison. The dummy model does not learn from the data; instead, it assigns a uniform probability to every character in the vocabulary. Its loss represents the expected performance of a system making entirely random predictions. The evaluation code is as follows:

```

@torch.no_grad()
def eval_dummy(path):
    # load file
    with open(path, 'r', encoding='utf-8') as f:
        test_text = f.read()
    # encode
    test_data = torch.tensor(encode(test_text), dtype=torch.long)

    # dummy model pred - uniform distribution over vocab
    losses = []
    uniform_logits = torch.zeros((1, vocab_size), device=device)
    # assign uniform probability across characters
    uniform_logits[:] = 1.0 / vocab_size
    model.eval()
    for i in range(0, len(test_data) - block_size, block_size):
        y = test_data[i+1:i+block_size+1].to(device)
        repeated = uniform_logits.repeat(block_size, 1) # repeat logits so shape matches expected (B*T, C)
        loss = F.cross_entropy(repeated, y) # compute cross entropy manually
        losses.append(loss.item())

    return sum(losses) / len(losses)

```

in which it loads and encodes the test file, then computes the cross-entropy loss between the true next characters and a fixed uniform logit distribution representing equal likelihood for all 40 characters. Because this uniform distribution is repeated at each position in the sequence, the computed loss corresponds to the expected performance of a model that makes purely random predictions. The final dummy loss was 3.6889, which served as a baseline: any trained model must achieve a loss less than this value to show that it has learnt meaningful structure from the child-speech data.

- (a) From the results obtained in Part I(c), Configuration 2 ($n_embd = 64$, $n_head = 4$, $n_layer = 4$) was identified as the best-performing model. It consistently achieved the lowest validation loss of the three reduced configurations and produced generated text that was syntactically cleaner and more representative of the child-speech data. Using this configuration, the test loss on `input_childSpeech_testSet.txt` was computed using the previously mentioned evaluation procedure.

The resulting test loss of 0.33018 is almost identical to the model's final validation loss (0.3378), indicating strong generalisation and little to no overfitting. A loss of this magnitude suggests that the model is accurately predicting the next characters in child-directed speech and has learnt the underlying transitional patterns and frequent word fragments present in the dataset. For comparison, the dummy baseline model achieved a much higher test loss of 3.6889, reflecting the performance of a system that assigns uniform probability across all characters. The trained model's substantially lower test loss shows that it has captured meaningful structure within the child-speech dataset rather than relying on chance-level predictions.

- (b) The trained model was also tested using the `input_shakespeare.txt` dataset, resulting in a test loss of 1.7284. Higher than the test loss on the child-speech dataset (0.3302) reported in ii(a), it indicates that the model performs significantly worse when predicting sequences from Shakespearean text. However, it outperforms the dummy baseline model, which achieves a loss of 4.1744, demonstrating that the trained model retains some general character-level knowledge even when working on text outside of its training domain.

The increase in loss relative to the child-speech test set can be attributed to several factors. The Shakespeare dataset has a larger character vocabulary (65 characters instead of 40), more complex syntax, and stylistic patterns that differ significantly from the linguistic structure present in the child-speech corpus. Since the model was trained exclusively on child-directed speech, its learnt distribution does not transfer well to the highly varied and archaic spelling, phrasing, and rhythmic patterns typical of Shakespeare. As a result, its next-character predictions become less accurate, increasing the loss.

From this outcome it becomes apparent that the model is specialised to the domain it was trained on and does not generalise strongly to linguistically distant datasets. In practice, such a training pipeline could be used to model, generate, or analyse sequences within a specific domain, such as child-directed speech, while emphasising the importance of domain adaptation or retraining for high-quality performance on distinct text sources, such as classical literature.

Appendix

```
import torch
import torch.nn as nn
from torch.nn import functional as F

'''
# hyperparameters - original
batch_size = 64 # how many independent sequences will we process in parallel?
block_size = 256 # what is the maximum context length for predictions?
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 384
n_head = 6
n_layer = 6
dropout = 0.2
'''

'''
# config1
batch_size = 64
block_size = 256
max_iters = 2000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 64
n_head = 4
n_layer = 2
dropout = 0.2
'''

'''
# config2 - (best/lowest val loss)
batch_size = 64
block_size = 256
max_iters = 2000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 64
n_head = 4
n_layer = 4
dropout = 0.2
'''

'''
# config3
batch_size = 64
block_size = 256
max_iters = 2000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 32
n_head = 2
n_layer = 2
```

```

dropout = 0.2
'''

# -----

torch.manual_seed(1337)

# check if using gpu
if (device == 'cuda'):
    print("Using GPU")
else:
    print("Using CPU")

# wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

#-----FILES-----
#file_name = 'input_childSpeech_trainingSet.txt'
#file_name = 'input_childSpeech_testSet.txt'
#file_name = 'input_shakespeare.txt'
with open(file_name, 'r', encoding='utf-8') as f:
    text = f.read()
#-----

# dataset size, its vocab of characters and its vocab size
dataset_size = len(text)

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)

# print some info about the dataset
print("-----")
print(f"file: {file_name}")
print(f"chars: {chars}")
print(f"dataset_size: {dataset_size}")
print(f"vocab_size: {vocab_size}")

# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

# Train and test splits
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out

#------(ii)-----
@torch.no_grad()

```

```

def eval_on_testset(path):
    # load file
    with open(path, 'r', encoding='utf-8') as f:
        test_text = f.read()
    # encode
    test_data = torch.tensor(encode(test_text), dtype=torch.long)

    losses = []
    model.eval()
    # break data into blocks of size block_size+store loss of each block
    for i in range(0, len(test_data) - block_size, block_size):
        x = test_data[i:i+block_size].unsqueeze(0).to(device)
        y = test_data[i+1:i+block_size+1].unsqueeze(0).to(device)
        _, loss = model(x, y)
        losses.append(loss.item())
    # retrain+return average loss
    model.train()
    return sum(losses) / len(losses)

#-----

@torch.no_grad()
def eval_dummy(path):
    # load file
    with open(path, 'r', encoding='utf-8') as f:
        test_text = f.read()
    # encode
    test_data = torch.tensor(encode(test_text), dtype=torch.long)

    # dummy model pred - uniform distribution over vocab
    losses = []
    uniform_logits = torch.zeros((1, vocab_size), device=device)
    # assign uniform probability across characters
    uniform_logits[:] = 1.0 / vocab_size
    model.eval()
    for i in range(0, len(test_data) - block_size, block_size):
        y = test_data[i+1:i+block_size+1].to(device)
        repeated = uniform_logits.repeat(block_size, 1) # repeat logits so shape matches expected (B*T, C)
        loss = F.cross_entropy(repeated, y) # compute cross entropy manually
        losses.append(loss.item())

    return sum(losses) / len(losses)

#-----

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # input of size (batch, time-step, channels)
        # output of size (batch, time-step, head size)
        B, T, C = x.shape
        k = self.key(x) # (B,T,hs)
        q = self.query(x) # (B,T,hs)
        # compute attention scores ("affinities")
        wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,hs)
        out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
        return out

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

```



```

def __init__(self, num_heads, head_size):
    super().__init__()
    self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
    self.proj = nn.Linear(head_size * num_heads, n_embd)
    self.dropout = nn.Dropout(dropout)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1)
    out = self.dropout(self.proj(out))
    return out

class FeedForward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in followup video
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)

```

```

        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)

    return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

model = GPTLanguageModel()
m = model.to(device)
# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

best_val = float('inf')

for iter in range(max_iters):
    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

        if losses['val'] < best_val:
            best_val = losses['val']
            torch.save(model.state_dict(), "best_model.pt")

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

# Load best model
model.load_state_dict(torch.load("best_model.pt"))
model.to(device)
model.eval()

#-----ii(a)-----

# Evaluate on child-speech test set
child_test_loss = eval_on_testset("input_childSpeech_testSet.txt")
print("Child speech TEST loss:", child_test_loss)

dummy_loss = eval_dummy("input_childSpeech_testSet.txt")
print("Dummy baseline test loss:", dummy_loss)

#-----ii(b)-----

```

```
# Evaluate on shakespeare test set
#shakespeare_test_loss = eval_on_testset("input_shakespeare.txt")
#print("Shakespeare TEST loss:", shakespeare_test_loss)

#dummy_loss = eval_dummy("input_shakespeare.txt")
#print("Dummy baseline test loss:", dummy_loss)

#-----
# generate from the model
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))
#open('more.txt', 'w').write(decode(m.generate(context, max_new_tokens=10000)[0].tolist()))
```